A Project II Report on

Plagiarism Detector

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Engineering in Software Engineering at Pokhara University

Submitted By:

NEON NEUPANE

BISHAL ACHARYA

NABIN GIRI

Department of Research and Development

GANDAKI COLLEGE OF ENGINEERING AND SCIENCE

Lamachaur, Kaski, Nepal

(Aug, 2026)

A Project II Report on

Plagiarism Detector

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Engineering in Software Engineering at Pokhara University

Submitted By:

NEON NEUPANE (22180031)

BISHAL ACHARYA (22180018)

NABIN GIRI (22180030)

Supervisor

ER. PRATIVA NYAUPANE

Department of Research and Development

GANDAKI COLLEGE OF ENGINEERING AND SCIENCE

Lamachaur, Kaski, Nepal

(Aug, 2026)

COPYRIGHT

SUPERVISOR RECOMMENDATION

This is to certify that this project report entitled Plagiarism Detector prepared and submitted by the below listed team of students in partial fulfillment of the requirement of the degree of Bachelor of Software Engineering awarded by Pokhara University has been prepared and completed under my supervision.

I hereby recommend the same for acceptance by the Gandaki College of Engineering and Science, Pokhara University.

1. Neon Neupane
2. Bishal Acharya
3. Nabin Giri

……...…

Er. Prativa Nyaupane
Supervisor
Department of Research and Development
Gandaki College of Engineering and Science
Pokhara University

Date: August 10, 2026

BONAFIDE CERTIFICATE

This is to certify that the project report entitled “Plagiarism Detector” is a bonafide record of the original work carried out by Neon Neupane, Bishal Acharya and Nabin Giri in partial fulfillment of the requirements for the degree of Bachelor of Engineering in Software Engineering.

The project was conducted under the guidance of Er. Prativa Nyaupane and the overall coordination of Er. Santosh Panth.

To the best of our knowledge, this work is original and has not been submitted to any other institution for the award of any degree or diploma.

Date of Evaluation: ……

….…….….	….…….….
Er. Prativa Nyaupane	Er. Santosh Panth
Supervisor	Project Head
Department of Research and Development	Research Management Committee

….…….…

External Supervisor

School of Engineering

ACKNOWLEDGEMENT

We would like to extend our sincerest gratitude to our project supervisor Er. Prativa Nyaupane for the valued suggestions and guidance given to us throughout the development of this project. The weekly review meetings kept the work honest and measurable, and many of the decisions recorded in this report, such as keeping a classical baseline for comparison and fixing the similarity threshold from experiment rather than by guess, came directly from those meetings.

We are thankful to the Department of Research and Development and to Er. Santosh Panth for providing us with the opportunity and the environment to build this system. We also thank the classmates and teachers who agreed to run their own assignments through the system during acceptance testing and who told us plainly what was confusing in the result screen.

Finally, we are grateful to the open source community behind Sentence-BERT, PyTorch, Django, React and Tesseract. Without these freely available tools a system of this kind could not have been built by three students in one semester.

Neon Neupane (22180031)

Bishal Acharya (22180018)

Nabin Giri (22180030)

ABSTRACT

Assignments in colleges are submitted digitally today, and copying a passage from the internet or from a classmate takes seconds. The tools that can detect this are either paid or restricted to institutions, and the free ones compare words rather than meaning, so a lightly reworded sentence escapes them. This project presents a free, browser based plagiarism detector that compares the meaning of sentences instead of their surface form. A batch of uploaded documents is compared against every other document in the batch, and any document can also be compared against public web pages. Text is read from PDF, DOCX, TXT and scanned images, the last through optical character recognition, and English as well as Nepali text is handled. Nothing is stored: each file is processed in memory and dropped once the answer is returned.

Two engines were built and compared. The baseline is a TF-IDF vector space model written directly with PyTorch tensors, and the proposed engine is a Sentence-BERT model, all-MiniLM-L6-v2, fine tuned with a Siamese architecture on thirty thousand labelled paraphrase pairs. On a held out split of the training corpus the fine tuned model reaches 82.6 percent accuracy against 74.3 percent for the pretrained model and 66.6 percent for the TF-IDF baseline at the same threshold. The gap between the score given to a paraphrased pair and the score given to an unrelated pair, which is what actually decides whether a copied sentence is caught, widened from 0.310 to 0.448 after fine tuning. Chunked, batched and cached encoding reduced the time taken to compare ten documents from 27.1 seconds to 5.3 seconds on the same machine.

Keywords: plagiarism detection, semantic similarity, Sentence-BERT, Siamese network, TF-IDF, optical character recognition, Devanagari, Django, React

TABLE OF CONTENTS

List of Figures

List of Tables

Chapter 1
INTRODUCTION

1.1 Background

Assignments are typed now. Lab reports, project documents, even the weekly write ups are all handed in as files, and that one change is what created the problem we set out to work on. A paragraph moves from a web page into a report in about four seconds, and the finished file carries no mark of where it came from. The teacher is then expected to spot the copy while marking forty submissions in a week. Nobody can do that reliably.

The tools that do the job properly are locked behind a price. Turnitin sells to institutions [11], not to a teacher who wants to check one batch of assignments on a Sunday evening. Quetext [12] and Copyleaks [13] hand out a small free quota and then ask for a card. What is left is the free checkers, and those match strings of words. We tested three of them before writing a line of code. Swap four words in a sentence, move a clause to the front, and the similarity they report drops to almost nothing, because the tool never read the sentence at all. It counted it. A student who copies on purpose figures this out in one attempt.

There is another way to do this, and it has only become practical recently. A transformer can turn a sentence into a vector so that two sentences meaning the same thing land near each other even when they share almost no words. Sentence-BERT [1] was built for exactly that, and its smaller variants run on an ordinary laptop with no graphics card, which mattered to us because none of us owns one. Our project takes that idea and wraps it in something a student or a teacher in Nepal can open in a browser, use immediately, and pay nothing for.

1.2 Problem Statement

No free tool within reach of a student or a teacher in Nepal catches a copied passage once it has been reworded. That is the short version. The free tools compare words, so the moment a sentence is paraphrased their score collapses and the copied work passes as original. The paid tools handle it properly, and they are out of reach on both counts, price and access.

Three more problems show up once you look at what actually gets submitted in a Nepali college. A good share of it is scanned, either handwritten or printed and photographed,

and a checker that only accepts typed text cannot read those files at all. Nepali is not handled either. A sentence in Devanagari ends with the danda and not the full stop, and the tokenizers behind these tools throw Devanagari words away before the comparison even starts. Then there is the question nobody asks until it is too late: the free services keep a copy of every file you give them, which is a strange thing to accept for unpublished student work.

So the gap we are filling is a checker that is free and open to anyone, that compares meaning instead of words, that reads the formats students really submit including scanned pages, that copes with Nepali as well as English, and that keeps nothing after it answers.

1.3 Objectives

1. To build a system that takes a batch of submitted documents, compares every one against every other, and reports for each pair how close they are and which sentences are the copied ones.
2. To check a document against public web pages as well, since a great deal of what gets copied never passes through another student at all.
3. To catch paraphrased text by fine tuning a Sentence-BERT model on labelled paraphrase pairs, and to measure it against a TF-IDF baseline we write ourselves, so that the improvement is shown rather than claimed.
4. To accept PDF, DOCX, TXT and scanned images, running the scans through optical character recognition, and to handle Devanagari alongside English.
5. To keep the whole thing free and open to anyone, with no login and no copy of any uploaded file kept once the answer has been sent.

1.4 How Our System Works

Five steps, and each one is its own module in the code. We split it that way so that any single step can be tested, replaced or explained to the panel without dragging the rest of the system into the conversation.

1. Reading the document

- The uploaded file is read in memory according to its type: directly for TXT and DOCX, through a PDF parser for PDF, and through Tesseract for images.
- A PDF that has no text layer, which is what a scanned document looks like, is detected automatically and sent to optical character recognition.

2. Preparing the text

- The text is cleaned, the spacing is normalised and it is split into sentences, ending a sentence at a full stop, a question mark, an exclamation mark or a danda.
- The sentence is the unit of comparison, because a report on the whole document would tell the teacher that something is wrong but not where.

3. Comparing every pair

- Each document is turned into a vector and every document is compared with every other one, which produces the similarity matrix.
- The user can choose the engine, that is, the fine tuned Sentence-BERT model or the TF-IDF baseline, and can move the threshold at which a sentence is called copied.

4. Checking against the web

- The most frequent meaningful words of the document are used as a search query, the result pages are opened, the navigation and the advertisements are removed and the remaining text is compared against the document.
- A page that cannot be fetched is skipped so that one dead link does not break the whole check.

5. Showing the result

- The similarity matrix, the percentage copied and the matched sentences are returned as JSON and drawn by the React application, which marks the copied sentences inside the document and explains the score bands in a legend.

Chapter 2
LITERATURE REVIEW

2.1 Theoretical Foundations

A. String and n-gram matching

The oldest method treats both documents as strings, or as sequences of word n-grams, and reports how much they share. It is fast, needs no training and never guesses, which explains why nearly every free checker still uses it. The weakness is built into the method rather than being a bug in some particular implementation. Nothing in it knows what a word means, so one synonym breaks the match. Potthast et al. [8] set out the evaluation framework used in the PAN competitions and show these systems falling away sharply once the copied passage has been obfuscated. Obfuscation is not the exception in student work. It is the normal case.

B. TF-IDF and the vector space model

Salton and Buckley [4] gave us term weighting. A document becomes a vector, each component is the frequency of a term weighted by how rare that term is across the collection, and two documents are compared by the cosine of the angle between their vectors. Reordering stops mattering and so does length, which is why the method has survived as the standard baseline in information retrieval for nearly forty years. We use it as our baseline engine and we wrote it ourselves with PyTorch tensors instead of importing it from a library. That was deliberate. Both engines then run on the same tensors and the same cosine, so any difference between them belongs to the model and not to somebody else's optimised library code.

C. Transformers and contextual embeddings

Vaswani et al. [10] introduced the transformer and Devlin et al. [2] built BERT on top of it, a model pre-trained on a large body of text that represents a word according to the words around it. BERT can certainly judge whether two sentences are related. The catch is that it wants both sentences fed in together. Comparing every pair in a batch of twenty documents would mean one model run per pair, and we did the arithmetic early enough to rule it out.

D. Sentence-BERT and the Siamese architecture

Reimers and Gurevych [1] fixed that with Sentence-BERT. Two copies of one network share their weights, each side encodes a single sentence into a fixed vector, and training pulls the vectors of a matching pair together while pushing a non-matching pair apart. The shared weights are what make the architecture Siamese. For us the useful consequence is arithmetic: encode each sentence once, then compare any two of them with a cosine, so a batch costs one encoding pass rather than one pass per pair. We use all-MiniLM-L6-v2, a distilled six layer model built on MiniLM [3]. It was chosen for a thoroughly unglamorous reason, which is that it runs on the laptops we own.

E. Optical character recognition

Plenty of what gets submitted is scanned rather than typed, so we needed an engine that reads images. Tesseract [9] is the open source one, and it takes language specific training data, which matters because Devanagari needs its own. Recognition on a clean scan is good. On a photograph taken at an angle in poor light it is not, and everything downstream inherits that, since the comparison can only be as good as the text that was read. We record it as a limitation rather than pretending otherwise.

F. Corpora for training and evaluation

Training this kind of model means finding labelled pairs, and there are fewer good options than we expected. The Quora Question Pairs corpus [6] carries roughly four hundred thousand pairs marked duplicate or not. The Microsoft Research Paraphrase Corpus [5], part of GLUE [15], holds sentence pairs pulled from news text and marked as paraphrase or not. Clough and Stevenson [7] assembled something closer to our actual situation, short answers copied deliberately at four different levels, but it runs to about a hundred documents and you cannot train on that. We train on Quora and then evaluate on both corpora, one the model has seen and one it has not, which turned out to be the most informative decision in the whole project.

2.2 Gap in Literature

Every technique above is well studied on its own. None of it reaches the student in Pokhara. The free checkers are still matching words. The academic work on semantic similarity stops at a sentence pair benchmark and is published as a score, not as something anybody can open, so it says nothing about reading a scanned PDF, nothing about comparing twenty submissions against each other, and nothing about how long that

takes on a five year old laptop. Devanagari text processing exists in its own literature and has not been joined to plagiarism detection. As for keeping an unpublished student document private, no free service we looked at treats it as a requirement at all.

Our contribution is engineering, not theory. We take a semantic similarity model that is already known to work on sentence pairs and deliver it as a finished, free and measured application: one that reads the formats students actually submit, copes with two scripts, compares a whole batch in a reasonable time, and keeps nothing afterwards.

2.3 Review of Related Works

Table 1: Review of the existing plagiarism checking tools

Tool / work	**Strengths**	**Limitations**
Turnitin [11]	Large private archive of past submissions; detects paraphrase; used by universities worldwide	Sold to institutions only; not reachable by an individual student or teacher; stores every submission
Quetext [12]	Free quota; simple interface; reports matched sources	Small free quota then paid; matching is based on repeated wording; no scanned input; no Nepali
Copyleaks [13]	Many file formats; API available	Paid after a trial; closed system; the document is uploaded to their servers
Free word matching checkers	No cost; instant	Report a copy only when the words are repeated; a reworded sentence is missed completely
Sentence-BERT [1]	Encodes a sentence once and compares by cosine; detects paraphrase; small variants run on a CPU	A model and a benchmark, not an application; nothing about file reading, batches, scripts or deployment
PAN evaluation work [8]	Standard corpora and measures for plagiarism detection	Research setting; obfuscated passages remain hard for surface methods
This project	Free and without login; semantic comparison of a whole batch and of web pages; PDF, DOCX, TXT and scanned images; English and Nepali; nothing stored	Trained on a general paraphrase corpus rather than on a corpus of student work; accuracy on Nepali lower than on English

Chapter 3

TOOLS AND METHODOLOGY

3.1 Required Tools

These are the tools we actually used, and the third column says why we picked each one rather than the alternative.

Table 2: Required tools

Tool	Purpose	Why it was chosen
Git and GitHub	Version control and collaboration	Three members working in parallel on backend, frontend and testing
Django and Django REST Framework	Backend and API	Mature Python framework, so the machine learning code and the web layer live in one language
PyTorch and sentence-transformers	Model training and inference	Sentence-BERT is published for this stack and runs on a CPU
React, Vite and Tailwind CSS	Frontend	Component model suits the matrix and highlight views; Vite gives a fast reload
Tesseract with pytesseract	Optical character recognition	Open source, offline, and has Devanagari training data
pdfplumber and python-docx	Reading PDF and DOCX	Handle multi column pages and give access to the raw paragraph text
BeautifulSoup and requests	Web scraping	Simple and reliable removal of scripts, navigation and advertisements
Visual Studio Code	Development environment	Extensions for Python, Django and React in one editor
draw.io and Matplotlib	Diagrams and charts	Diagrams for the design, charts drawn from the measured results

3.2 Methodology

3.2.1 System Development Approach

We worked in one week sprints. The requirement was clear from day one, but the design was not, and on the machine learning side it could not have been: which model, which

corpus, which threshold, none of these can be argued into place and all of them had to be settled by running something and looking at the number that came back. Short sprints with a review at the end suited that. Ten sprints ran between 8 June 2026 and 10 August 2026. Each one closed with a meeting with our supervisor, and the minute of that meeting records what was finished and what was assigned for the week ahead.

3.2.2 Implementation Process

Phase 1: Design (sprints 1 and 2)

- Fixed the scope to two checks, the batch comparison and the web comparison, and fixed the supported file formats.
- Selected the model, the training corpus and the technology stack, and wrote the plan document into the repository.
- Drew the data flow, use case and sequence diagrams and froze the API contract so that the frontend and the backend could be built in parallel.

Phase 2: Core development (sprints 3 to 6)

- Set up the Django project, the detector application and the services package, and the React project with Tailwind.
- Implemented text extraction with optical character recognition and the preprocessing utilities.
- Trained the model and implemented both similarity engines behind one interface.
- Built the file uploader, the similarity matrix and the highlight components.

Phase 3: Integration (sprints 7 and 8)

- Wrote the web scraping service with a fallback for pages that cannot be fetched, and the sentence level highlight logic.
- Exposed the three API endpoints and connected the frontend to them through a single API module.
- Tested the complete flow with real documents and presented the progress.

Phase 4: Quality (sprints 9 and 10)

- Removed the slowness of the comparison by chunking, batching and caching the encoding, and made the result usable on a small screen.
- Added Devanagari support, clear error messages for every failure case, the engine and threshold controls, the unit tests and the documentation.

3.2.3 Technologies Used

Table 3: Technologies used in the system

Layer	**Technology**
Frontend	React 18, Vite 5, Tailwind CSS 3
Backend	Python 3.12, Django 5, Django REST Framework
Machine learning	PyTorch, sentence-transformers, all-MiniLM-L6-v2 fine tuned with a Siamese architecture
Text extraction	pdfplumber, python-docx, Pillow, Tesseract through pytesseract
Web checking	requests, BeautifulSoup, DuckDuckGo HTML endpoint
Storage	None. Every uploaded file is processed in memory and dropped

3.2.4 System Architecture

Three layers. At the top a React application draws the upload screen, the similarity matrix and the highlighted result, and it reaches the server through exactly one module, so every line of network code in the frontend lives in one file called api.js. Below that sits a Django REST Framework service with three endpoints. Its views are thin on purpose: read the request, call a service, return JSON, nothing else. The logic all lives in the service layer, one responsibility per module, which gives us text extraction, preprocessing, similarity, highlighting and web scraping as five separate files. There is no data layer at all. Nothing is stored, so there is nothing to store it in.

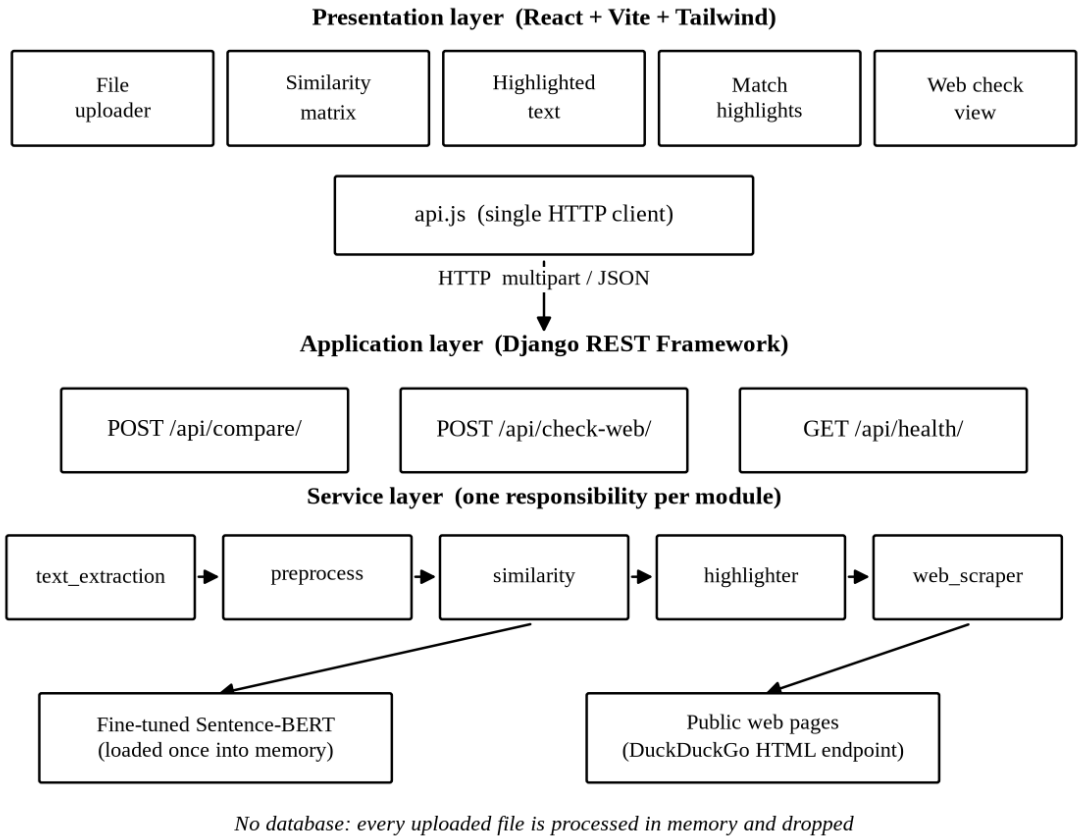


Figure 1: System architecture of the Plagiarism Detector

3.2.5 Agile Implementation

Every sprint ended the same way. We demonstrated what was working, the achievements of each member went into the minute, and the tasks for the following week were handed out before anyone left the room. The table lists what each sprint delivered. The commit history in the repository follows the same order and carries the same dates, so anything claimed in this table can be checked against the code rather than taken on trust.

Table 4: Sprint deliverables of the ten weekly sprints

Sprint	Week ending	Delivered
1	08 Jun 2026	Scope frozen, module division, roles and the working plan
2	15 Jun 2026	Model, corpus and stack selected; repository created; plan and pipeline documents committed
3	22 Jun 2026	Data flow, use case and sequence diagrams; API contract frozen; sample documents

4	29 Jun 2026	Django project, detector application and services package; React project with Tailwind; file uploader
5	06 Jul 2026	Text extraction with OCR, preprocessing utilities, similarity matrix component, training script
6	13 Jul 2026	Model fine tuned; both engines implemented; highlight components; scraping tested in a notebook
7	20 Jul 2026	Web scraping service with fallback, highlight logic, three API endpoints, frontend connected
8	27 Jul 2026	End to end testing, web check result view, README and progress presentation
9	03 Aug 2026	Chunked, batched and cached encoding; responsive result screens; project guide
10	10 Aug 2026	Devanagari support, error handling, engine and threshold controls, unit tests, user manual

3.2.6 Designs

Use Case Diagram

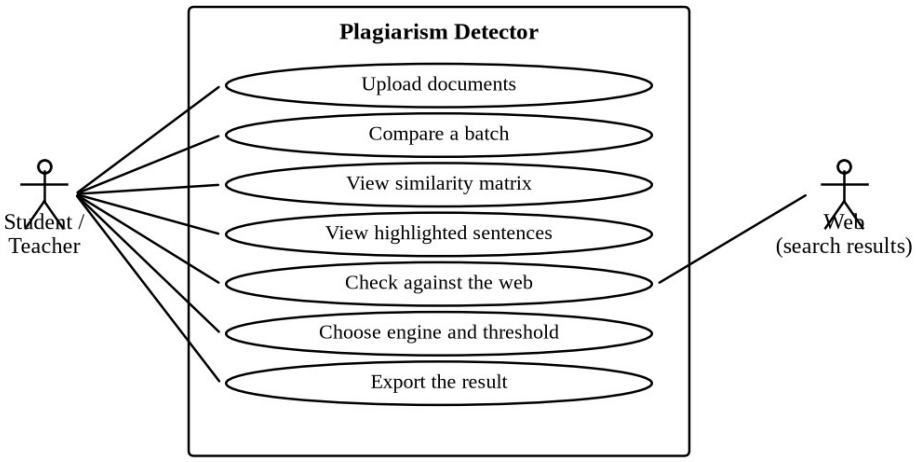


Figure 2: Use case diagram of the system

UC1: Upload documents

Actor: Student or teacher

Preconditions: The application is open in a browser.

Postconditions: The selected files are held in the browser, ready to be sent.

Flow: The user drags the files onto the upload box or selects them; the type and the size of each file are validated; invalid files are refused with a message.

UC2: Compare a batch

Actor: Student or teacher

Preconditions: At least two readable documents have been selected.

Postconditions: A similarity matrix and the flagged pairs are displayed.

Flow: The user presses Compare; the files are sent to the compare endpoint; the text is extracted, cleaned, split into sentences and encoded; every pair is scored and the pairs above the threshold are highlighted.

UC3: View the highlighted sentences

Actor: Student or teacher

Preconditions: A comparison has produced at least one flagged pair.

Postconditions: The copied sentences are marked inside the text of the document.

Flow: The user opens a flagged pair; each sentence of the first document is shown with its best match in the second and with its score; the sentences above the threshold are marked.

UC4: Check against the web

Actor: Student or teacher

Preconditions: One document or a pasted text is provided.

Postconditions: A list of matching pages with their links and scores is displayed.

Flow: The keywords of the document are extracted and searched; the result pages are fetched and cleaned; each page is compared with the document; a page that cannot be fetched is skipped.

UC5: Choose the engine and the threshold

Actor: Student or teacher

Preconditions: The application is open.

Postconditions: The next comparison uses the selected engine and threshold.

Flow: The user selects Sentence-BERT or TF-IDF and moves the threshold slider; the values are sent with the request and are echoed back in the response.

UC6: Export the result

Actor: Student or teacher

Preconditions: A comparison has been completed.

Postconditions: A printable report of the result is produced.

Flow: The user presses the export control; the matrix, the scores and the highlighted sentences are laid out for printing and the browser print dialogue is opened.

Data Flow Diagrams

The level 0 diagram shows the system as one process between the user and the public web. The level 1 diagram opens that process into the five services and shows that the model is used by the comparison, the highlighting and the web check, and that no data store appears anywhere in the diagram.

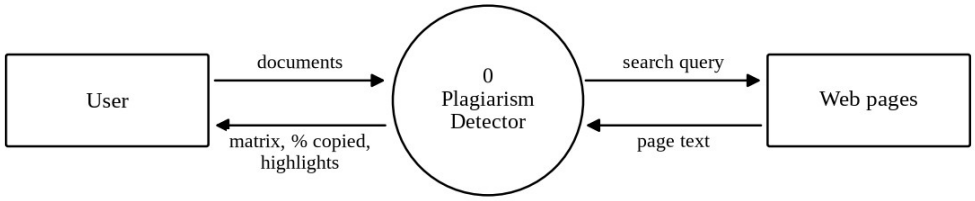


Figure 3: Level 0 data flow diagram (context diagram)

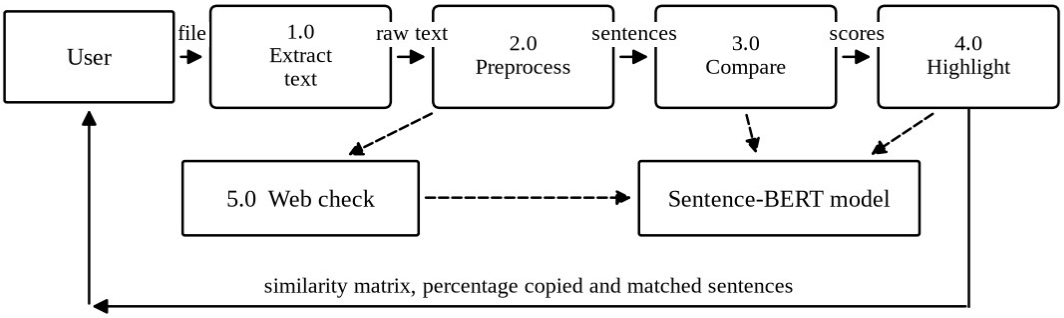


Figure 4: Level 1 data flow diagram

Sequence Diagram

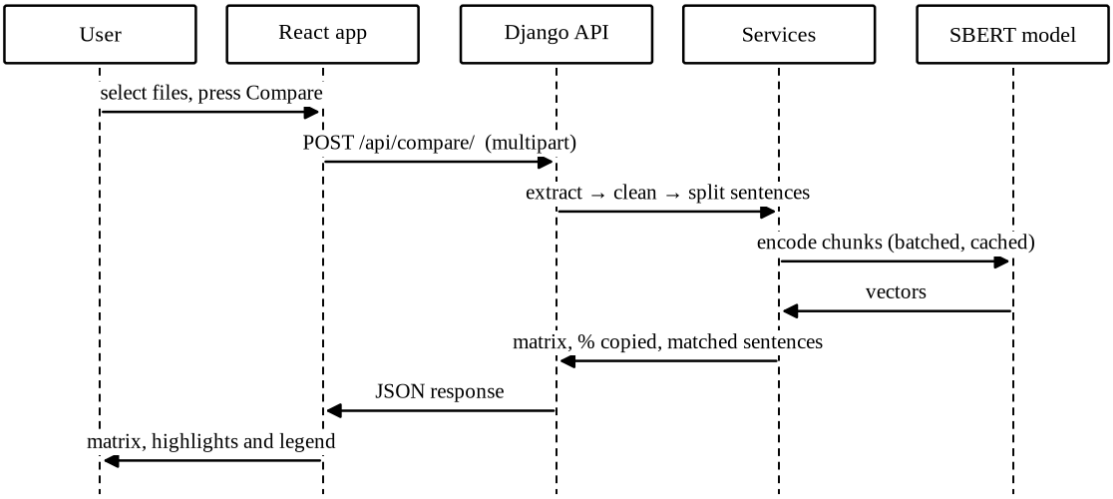


Figure 5: Sequence diagram of a batch comparison

Timeline

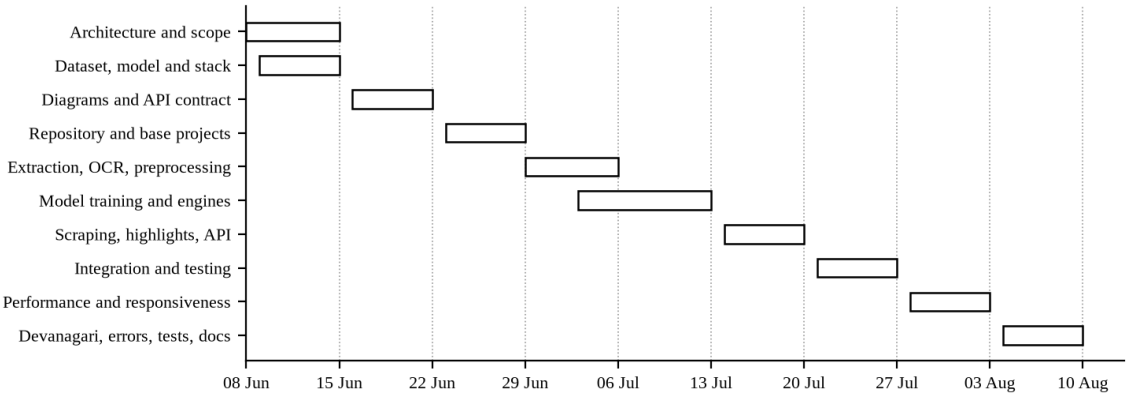


Figure 6: Timeline chart of the ten weekly sprints

3.2.7 The Model and its Training

Our engine is a Sentence-BERT model built on all-MiniLM-L6-v2, a six layer distilled transformer that turns any piece of text into a vector of three hundred and eighty four numbers. Two copies of that network with shared weights give us the Siamese arrangement. Each side encodes one sentence, the cosine of the two vectors is compared against the label of the pair through a cosine similarity loss, and training drags matching pairs together while pushing the rest apart.

Training runs from a single script, train.py, which downloads the corpus, fine tunes the model and writes the checkpoint that the backend then picks up on its own. We took

thirty thousand labelled pairs from the Quora Question Pairs corpus [6] and split them twenty seven thousand for training against three thousand held back. One epoch, batch size thirty two, warm up over the first ten percent of the steps. On the held out split the fine tuned model scored an accuracy of 0.800 at the 0.70 threshold. Chapter 5 carries the full evaluation, including the baseline and a second corpus the model had never seen.

Two decisions here are worth writing down. First, we kept the TF-IDF baseline inside the shipped system instead of deleting it once the trained model worked. Without it, the claim that our model detects paraphrase would rest on nothing anyone could check. Second, a document is never encoded in one call. The model quietly truncates anything longer than its input window, which meant the last part of a long report was not being compared at all and nothing in the output said so. We found it while measuring speed, not while testing accuracy. A document is now cut into chunks of five sentences, the chunks go through in batches, and the document vector is the mean of its chunk vectors.

3.2.8 The Application Programming Interface

We froze this contract in sprint 3 and never changed it. That is the only reason two people could build the frontend while a third built the backend without anyone waiting on anyone.

Table 5: API endpoints of the system

Endpoint	Method	Request	Response
/api/compare/	POST	files, method, threshold	documents, matrix, flagged_pairs with percent_copied and matches, threshold, skipped
/api/check-web/	POST	one file or text, optional urls, threshold	document, sources with url, score, percent_copied and matches, skipped
/api/health/	GET	none	status, model_fine_tuned

3.2.9 Handling of Devanagari Text

Nepali is not English in a different alphabet, and assuming it was cost us a sprint. Three things had to change. A sentence in Devanagari closes with the danda, so the splitter now treats that character as an ending whether or not a space follows it, and quite often there is no space. The tokenizer behind the baseline engine matched Latin letters and digits only,

which meant it returned an empty list for Nepali text and reported a similarity of zero for two identical Nepali documents. We widened its pattern to the Devanagari range and added the common Nepali function words to the stopword list. Tesseract gets the Nepali training data when the machine has it and falls back to English when it does not, and that check happens once and is then remembered.

Chapter 4
TESTING

We tested at three levels. Unit tests cover the services and run automatically. The whole system was then walked through by hand against a list of functional cases. Last, we handed the finished application to classmates and teachers and watched them use it on their own work, which is where the most useful criticism came from.

4.1 Test Objectives

- Check that every supported format is read correctly, and that a file we cannot read produces a sentence the user understands rather than a 500 page.
- Check that sentence splitting works in English and in Devanagari, the danda included.
- Check that a copied or reworded document scores above an unrelated one, and that any document compared with itself comes back as exactly one.
- Check that nothing falls over on an empty input, a corrupt file or a web page that refuses to answer.
- Check that somebody who has never met us can read the result screen and know what it is telling them.

4.2 Unit Testing

There are twenty four unit tests, written with the standard library runner and kept in the tests folder of the backend. None of them start Django and none of them load the trained model, which is why the whole suite finishes in well under a second and we could afford to run it after every change instead of once a week. All twenty four pass.

Table 6: Unit tests written for the services

Module	Tests	What is checked
preprocess	9	Cleaning and lowercasing; stopwords removed; Devanagari words kept; sentence splitting in English; the danda with and without a following space; short fragments ignored; keyword extraction
text_extraction	7	Plain text and UTF-8 Devanagari read correctly; empty file, unsupported

		extension, unreadable text and corrupt file each raise a clear message; the OCR language string is valid
similarity	8	A document is identical to itself; the matrix is symmetric; a copied document scores higher than an unrelated one; empty documents do not crash; chunking splits a long document into chunks of five sentences

4.3 Functional Test Cases

Table 7: Functional test cases

ID	Purpose	Test case	Result
TC1	Upload	No file selected	Compare button stays disabled
TC2	Upload	Only one document uploaded	Upload at least 2 readable documents message
TC3	Upload	Unsupported file such as .zip	Unsupported file type message with the list of accepted types
TC4	Upload	Empty file uploaded	The file is empty message
TC5	Upload	Corrupt DOCX file	The file could not be read message
TC6	Extraction	Scanned image with clear text	Text read through OCR and compared normally
TC7	Extraction	Scanned page of very poor quality	No text could be read message; the other files are still compared
TC8	Comparison	Two identical documents	Score of 1.00 and 100 percent copied
TC9	Comparison	Original and its paraphrase	Flagged, with the reworded sentences marked
TC10	Comparison	Two unrelated documents	Score near zero, not flagged
TC11	Comparison	Engine switched to TF-IDF	The matrix is recalculated with the baseline scores
TC12	Comparison	Threshold moved to 0.50	More pairs are flagged and the legend shows

			the new value
TC13	Nepali	Nepali document with danda punctuation	Sentences split correctly and compared
TC14	Web check	Pasted text with no URL given	Keywords searched and matching pages listed with their scores
TC15	Web check	URL that does not respond	The page is skipped and the remaining pages are still reported

4.4 Acceptance Testing

We gave the deployed system to classmates and to two teachers and asked them to use their own assignments, not our prepared samples. Nothing crashed. The complaint we heard again and again was that the score meant nothing to them: they could see 0.67 on the screen and had no idea whether that was bad. The legend explaining the score bands exists because of those sessions. The second comment came from both teachers independently, which is that they would not act on a number without first seeing the sentence behind it. That settled an argument we had been having internally and confirmed that reporting at sentence level, not only at document level, was the right call.

Chapter 5
RESULT AND DISCUSSION

5.1 Output of the System

The sample set in the repository holds four files: an original passage about photosynthesis, a paraphrase of it with almost every word swapped, an unrelated passage and a Nepali document. Putting all four through the system produces the scores below. Both engines separate the paraphrased pair from the unrelated ones, so at first glance both look fine. The size of the gap is what actually matters. The baseline scores the paraphrase at 0.47, and it only gets that far because a handful of words survived the rewriting. The trained model sees that the two passages are saying the same thing.

Table 8: Similarity scores produced for the sample documents

Pair	TF-IDF	Sentence-BERT	Percentage copied
Original and its paraphrase	0.4676	0.6676	25.0 %
Original and the unrelated passage	0.0000	0.0062	0.0 %
Original and the Nepali document	0.0000	-0.0181	0.0 %
Paraphrase and the unrelated passage	0.0000	0.0967	0.0 %

One number here surprised us. The same pair scores 0.8960 under the pretrained model and 0.6676 under our fine tuned one, which looks at first like the training made things worse. It did not. The scale moved. Section 5.4 works through what happened, because it decides which threshold belongs with which model.

5.2 Accuracy of the Two Engines

We evaluated on two labelled sets rather than one. The first is the validation split of the Quora corpus, the same corpus the model trained on, which tells us whether the training worked at all. The second is the validation split of the Microsoft Research Paraphrase Corpus, which the model had never seen and which is drawn from news text rather than forum questions. That second set is the honest test, because a plagiarism checker meets text it was not trained on every single time somebody uses it. Both were scored at the fixed threshold of 0.70.

Table 9: Accuracy of the engines on the two evaluation sets

Evaluation set	Engine	Accuracy	Precision	Recall	F1
Quora validation (2000 pairs)	TF-IDF baseline	0.6665	0.5380	0.2558	0.3467
	Sentence-BERT pretrained	0.7430	0.5800	0.9321	0.7151
	Sentence-BERT fine tuned	0.8260	0.7544	0.7370	0.7456
MRPC validation (408 pairs)	TF-IDF baseline	0.5049	0.9326	0.2975	0.4511
	Sentence-BERT pretrained	0.7475	0.7733	0.8925	0.8286
	Sentence-BERT fine tuned	0.6691	0.8077	0.6774	0.7368

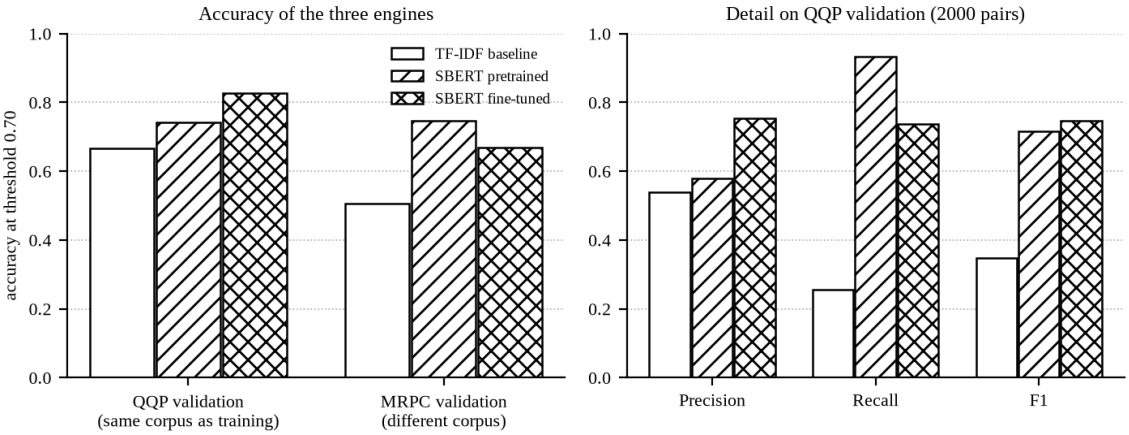


Figure 7: Accuracy of the three engines on both evaluation sets

Three things stand out in that table. Fine tuning worked. On the corpus it trained on, accuracy went from 0.7430 to 0.8260, a gain of 8.3 points, and the 0.70 threshold we had fixed weeks earlier by experiment turned out to be the best available threshold for that model on that corpus, which was luck as much as judgement. The second result is harsher. Our baseline does not merely lag behind, it fails at the one job this project exists to do. Its recall on paraphrased pairs is 0.2558 on the first set and 0.2975 on the second. Read that as roughly three reworded passages in every four walking straight past it. Third, the separation between the classes, meaning the distance between the average score of a paraphrased pair and the average score of an unrelated one, widened on both sets after

training, from 0.310 to 0.448 on the first and from 0.127 to 0.173 on the second. Separation is the measure we care about most, because it is what gives a threshold room to sit in.

5.3 Confusion Matrix

The counts behind those accuracies are below, and they describe the character of each model better than the accuracy column does. The pretrained model says yes too readily and produces 467 false positives. Ours refuses far more of them, 166, and pays for that by missing some genuine pairs. Which behaviour you want depends entirely on what a mistake costs, and in our case a false positive is an accusation aimed at a student who did nothing wrong. We prefer the cautious model. That preference only holds because the output is read as a similarity measurement and the final judgement stays with the teacher, which is a position we take seriously enough to repeat in the user manual.

Table 10: Confusion matrix of the three engines on the in-domain set

Engine	**True positive**	**True negative**	**False positive**	**False negative**
TF-IDF baseline	177	1156	152	515
Sentence-BERT pretrained	645	841	467	47
Sentence-BERT fine tuned	510	1142	166	182

5.4 The Threshold

The threshold decides everything a user ever sees, and we had been treating it as though it were a property of the problem. It is not. On the corpus the model trained on, the best threshold for our fine tuned model is 0.7, the value already in the code. Move to the second corpus and the best threshold for the very same model drops to 0.45. Held at 0.70 it scores 0.6691 there, against 0.7328 once the threshold is tuned. The baseline shows the same thing far more violently, because its scores live on a completely different scale: its best threshold on the second corpus is 0.25, and moving to it lifts accuracy from 0.5049 to 0.7108.

That finding changed the product. The threshold is a slider in the finished interface instead of a constant buried in a Python file, and the legend on the result screen always prints the value that was actually used. The default of 0.70 is right for our model on text resembling what it trained on. For anything else, the user needs to be able to reach in and move it.

5.5 Speed of the Comparison

Our first working version encoded a whole document in one call, then encoded that document's sentences all over again for every pair it appeared in. With ten documents that is the same work done nine times. The final version cuts each document into chunks of five sentences, pushes them through in batches, and keeps every encoded sentence in a cache, so a distinct sentence is encoded once per request and never again. We measured both versions on one machine, on the processor rather than the graphics card, with 10 documents of about 2628 words each, 1335 sentences altogether, and all 45 pairs highlighted in both runs so that the only difference between them is the encoding work.

Table 11: Comparison time before and after the optimisation

Version	Time taken	Notes
Before	27.1 seconds	Whole document in one call; sentences re-encoded for every pair; the tail of a long document silently dropped
After	5.3 seconds	Chunked into five sentence blocks, encoded in batches, every sentence cached; the whole document is used
Improvement	5.1 times faster	Same documents, same pairs, same machine

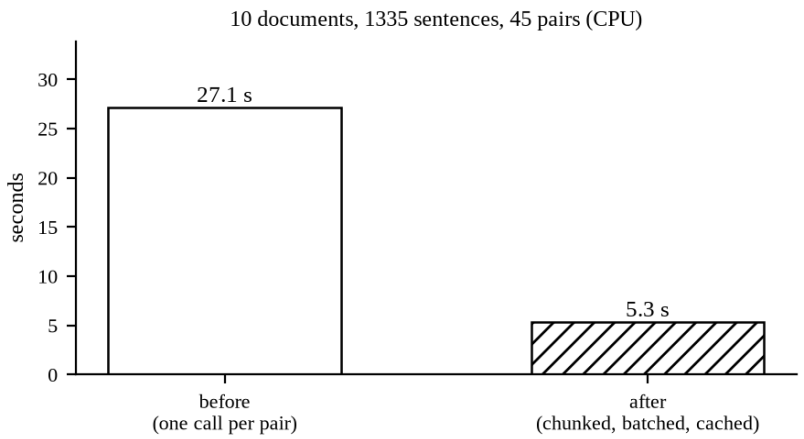


Figure 8: Comparison time before and after the optimisation

5.6 Impact and Significance

What this gives a teacher in a Nepali college used to require an institutional subscription. Drop a batch of submissions in, scanned ones included, and see within seconds which files resemble each other and which sentences are doing the resembling. The comparison

is semantic, so the standard defence of rewording a copied paragraph stops working, and section 5.2 puts a number on how much it stops working: the word based baseline catches roughly one paraphrased pair in four, ours catches about three in four.

The privacy decision matters as much as any accuracy figure in this report. No uploaded file is written to disk or to a database at any point in the request, so a student can check an unpublished report without handing a copy of it to a company that will keep it indefinitely. Every component is open source and the whole thing runs on an ordinary laptop with no graphics card, which means a department can host it for its own students and pay nothing for the privilege.

5.7 Limitations

- Nepali accuracy sits below English accuracy. We adapted the text handling for Devanagari but not the model, which still learned from English paraphrase pairs only.
- The best threshold for one kind of text is not the best for another, as section 5.4 shows. We put the control in the interface, but the user still has to understand what they are moving.
- The web check only sees what it can reach at that moment. A blocked or slow page gets skipped, which keeps the system alive and also means the check is never exhaustive.
- A poor scan gives poor text out of the recognition step, and no amount of model quality downstream can repair that.
- We compare against the documents handed to us and against public web pages. There is no archive of past submissions, so a paragraph lifted from a report handed in three years ago goes straight through.
- Source code, mathematical notation and figures are not handled at all.

Chapter 6
CONCLUSION AND FUTURE WORKS

We set out to build a free plagiarism detector that compares meaning instead of words. That is done, and unlike most claims of this sort it has been measured. The finished system takes PDF, DOCX, TXT and scanned images in English and in Nepali, compares a batch of documents against each other and any single document against public web pages, returns a similarity matrix along with the percentage copied, and marks the offending sentences inside the text itself. No login, no cost, nothing kept afterwards.

The central claim rests on evidence, not on assertion, and that was the point of keeping the baseline alive inside the system. We fine tuned a Sentence-BERT model with a Siamese architecture on thirty thousand labelled pairs and put it against a TF-IDF baseline written for the same codebase. On the training corpus, accuracy climbed from 0.7430 for the pretrained model to 0.8260 for ours, while the baseline managed 0.6665 and recovered only 0.2558 of the paraphrased pairs. We held the engineering work to the same standard. Chunked, batched and cached encoding took the comparison of ten documents from 27.1 seconds down to 5.3, and while doing it uncovered a defect nobody had noticed, where the tail of a long document was never being compared at all.

One result caught us out, and it is the one we would defend most warmly. Fine tuning does not simply lift every number on the page. It moves the scale the model scores on, which means a threshold fixed against one corpus is quietly wrong for another. We only saw it because we evaluated on a corpus the model had never met. The finding changed the product: the threshold left the source code and became a control in the interface, and the value actually used is now printed beside every result.

Future Works

6. Train on a corpus of real student work, such as the Clough and Stevenson collection [7], so that the model sees the kind of copying that actually happens in assignments rather than duplicate questions from a web forum.
7. Fine tune a multilingual model on Nepali paraphrase pairs so that the accuracy on Devanagari text matches the accuracy on English.

8. Add an archive of past submissions for a department, so that a document can be checked against the work of previous years, which is where a large part of copying comes from.
9. Add support for source code, where plagiarism is normally detected by comparing structure rather than text.
10. Add an account for teachers who want to keep the generated reports, keeping the anonymous mode as the default so that the privacy guarantee is not lost.

BIBLIOGRAPHY

[1] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP), Hong Kong, 2019, pp. 3982–3992.

[2] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in Proceedings of NAACL-HLT, Minneapolis, 2019, pp. 4171–4186.

[3] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang and M. Zhou, “MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers,” in Advances in Neural Information Processing Systems 33, 2020.

[4] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” Information Processing and Management, vol. 24, no. 5, pp. 513–523, 1988.

[5] W. B. Dolan and C. Brockett, “Automatically Constructing a Corpus of Sentential Paraphrases,” in Proceedings of the Third International Workshop on Paraphrasing, 2005.

[6] S. Iyer, N. Dandekar and K. Csernai, “First Quora Dataset Release: Question Pairs,” Quora, 2017. [Online]. Available: https://quoradata.quora.com/. [Accessed 2 Aug 2026].

[7] P. Clough and M. Stevenson, “Developing a corpus of plagiarised short answers,” Language Resources and Evaluation, vol. 45, no. 1, pp. 5–24, 2011.

[8] M. Potthast, B. Stein, A. Barrón-Cedeño and P. Rosso, “An Evaluation Framework for Plagiarism Detection,” in Proceedings of the 23rd International Conference on Computational Linguistics (COLING), Beijing, 2010, pp. 997–1005.

[9] R. Smith, “An Overview of the Tesseract OCR Engine,” in Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR), Curitiba, 2007, pp. 629–633.

[10] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser and I. Polosukhin, “Attention Is All You Need,” in Advances in Neural Information Processing Systems 30, 2017, pp. 5998–6008.

[11] Turnitin LLC, “Turnitin Similarity,” 2026. [Online]. Available: https://www.turnitin.com/. [Accessed 12 Jun 2026].

[12] Quetext, “Quetext Plagiarism Checker,” 2026. [Online]. Available: https://www.quetext.com/. [Accessed 12 Jun 2026].

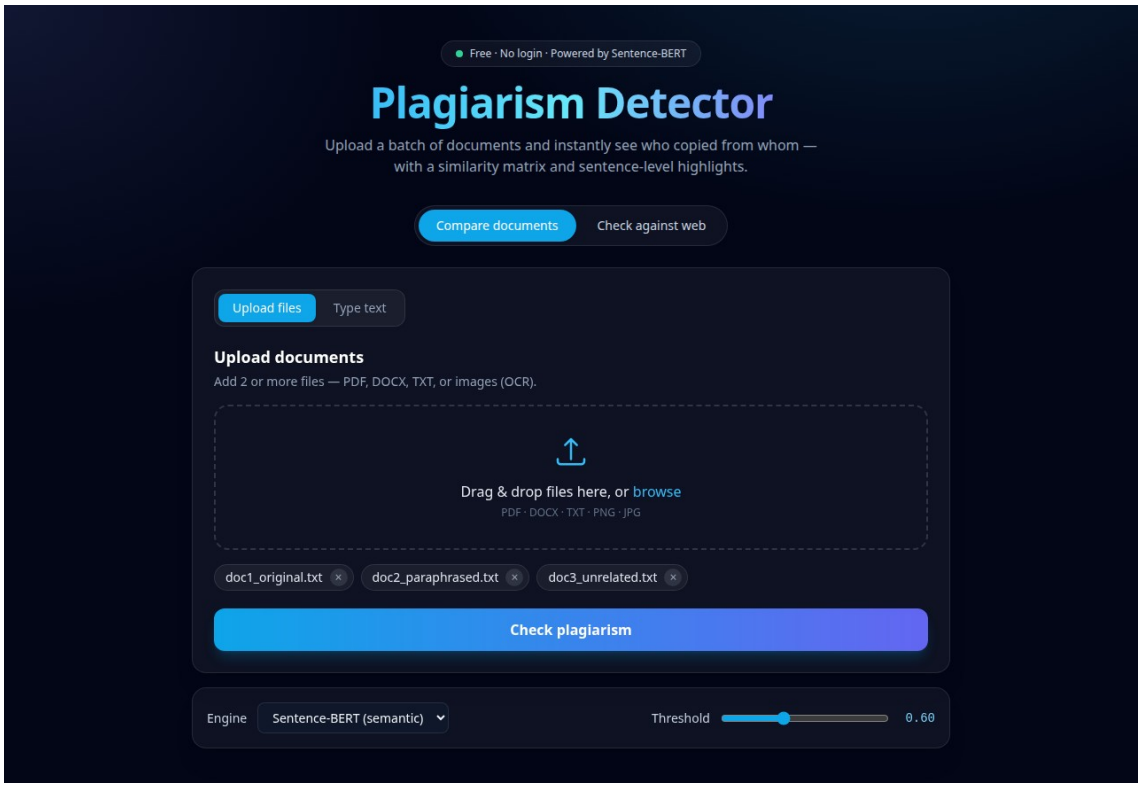
[13] Copyleaks, “Plagiarism and AI Content Detector,” 2026. [Online]. Available: https://copyleaks.com/. [Accessed 12 Jun 2026].

[14] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy and S. R. Bowman, “GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding,” in Proceedings of ICLR, 2019.

[15] Encode OSS Ltd., “Django REST Framework,” 2026. [Online]. Available: https://www.django-rest-framework.org/. [Accessed 20 Jun 2026].

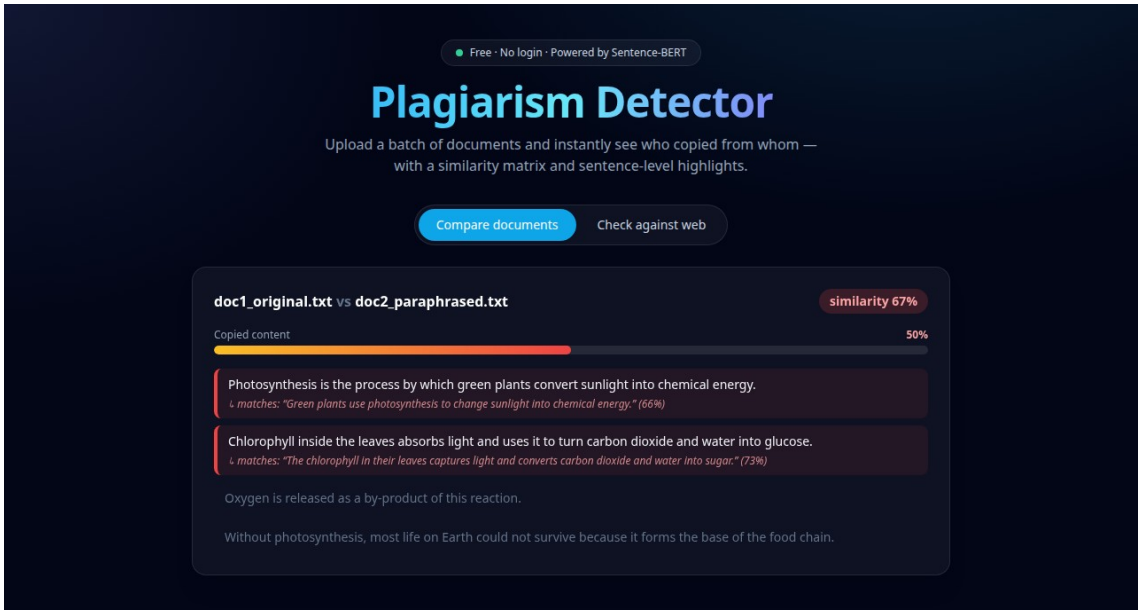
ANNEXES

Annex 1: Upload screen



Annex 2: Similarity matrix and summary

Annex 3: Highlighted sentences of a flagged pair



Annex 4: Web check result